Formal Mathematics with Lean and AI: Lecture 3

Overview

Last time:

- Sequent calculus ($\Gamma \vdash F$) and lambda calculus for proofs
- Curry–Howard correspondence (propositions as types)

Overview

Last time:

- Sequent calculus ($\Gamma \vdash F$) and lambda calculus for proofs
- Curry–Howard correspondence (propositions as types)
- Implication ($\rightarrow$ as $\rightarrow$)

Overview

Last time:

- Sequent calculus ($\Gamma \vdash F$) and lambda calculus for proofs
- Curry–Howard correspondence (propositions as types)
- Implication ($\to$ as $\to$)
- Conjunction ($\wedge$ as $\times$), negation ($\neg P := P \to \mathrm{False}$)

Overview

Last time:

- Sequent calculus ($\Gamma \vdash F$) and lambda calculus for proofs
- Curry–Howard correspondence (propositions as types)
- Implication ($\to$ as $\to$)
- Conjunction ($\wedge$ as $\times$), negation ($\neg P := P \to \mathrm{False}$)
- Universal quantifier ($\forall$) and dependent types ($\prod$)

Today:

- Disjunction ($\vee$): our first **inductive type**
- Natural numbers, recursion principle, pattern matching
- Defining addition on natural numbers
- Equality: Leibniz equality and inductive equality

Disjunction: Deduction Rules

We have rules for $\rightarrow$, $\wedge$, $\forall$, $\neg$, but we are missing $\vee$.

Introduction rules (two ways to prove $A \vee B$):

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B}$$

Disjunction: Deduction Rules

We have rules for $\rightarrow$, $\wedge$, $\forall$, $\neg$, but we are missing $\vee$.

Introduction rules (two ways to prove $A \vee B$):

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B}$$

Elimination rule (case analysis):

$$\frac{\Gamma \vdash A \vee B \qquad \Gamma \cup \{A\} \vdash C \qquad \Gamma \cup \{B\} \vdash C}{\Gamma \vdash C}$$

Disjunction: Deduction Rules

We have rules for $\to$, $\wedge$, $\forall$, $\neg$, but we are missing $\vee$.

Introduction rules (two ways to prove $A \vee B$):

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B} \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B}$$

Elimination rule (case analysis):

$$\frac{\Gamma \vdash A \vee B \qquad \Gamma \cup \{A\} \vdash C \qquad \Gamma \cup \{B\} \vdash C}{\Gamma \vdash C}$$

To *use* a proof of $A \vee B$, we must handle both cases.

Disjunction in Lambda Calculus: Sum Types

Recall: $A \wedge B$ corresponds to a pair (x, y) with $x : A$ and $y : B$.

$A \vee B$ corresponds to a **sum type** $A + B$:

- $\mathrm{inl}(a) : A + B \quad$ (given $a : A$)
- $\mathrm{inr}(b) : A + B \quad$ (given $b : B$)

Disjunction in Lambda Calculus: Sum Types

Recall: $A \wedge B$ corresponds to a pair (x, y) with $x : A$ and $y : B$.

$A \vee B$ corresponds to a **sum type** $A + B$:

- $\mathrm{inl}(a) : A + B \quad$ (given $a : A$)
- $\mathrm{inr}(b) : A + B \quad$ (given $b : B$)

Elimination: given $p : A + B, f : A \to C, g : B \to C$:

$$\mathrm{match}\ p\ \mathrm{with}\ \mathrm{inl}(a) \Rightarrow f(a) \mid \mathrm{inr}(b) \Rightarrow g(b) \quad : \ C$$

Disjunction in Lambda Calculus: Sum Types

Recall: $A \wedge B$ corresponds to a pair (x, y) with $x : A$ and $y : B$.

$A \vee B$ corresponds to a **sum type** $A + B$:

- $\mathrm{inl}(a) : A + B \quad$ (given $a : A$)
- $\mathrm{inr}(b) : A + B \quad$ (given $b : B$)

Elimination: given $p : A + B, f : A \to C, g : B \to C$:

$$\mathrm{match}\ p\ \mathrm{with}\ \mathrm{inl}(a) \Rightarrow f(a) \mid \mathrm{inr}(b) \Rightarrow g(b) \quad : \ C$$

In Lean:

`Or.inl h` and `Or.inr h` to introduce,
`Or.rec` or `match` to eliminate.

Disjunction as an Inductive Type

We can *define* $A \vee B$ by declaring its constructors:

```
inductive Or (A B : Prop) : Prop where
  | inl : A → Or A B
  | inr : B → Or A B
```

Disjunction as an Inductive Type

We can *define* $A \vee B$ by declaring its constructors:

```
inductive Or (A B : Prop) : Prop where
  | inl : A → Or A B
  | inr : B → Or A B
```

This is an **inductive definition**: we list all the ways to build an element.

The system automatically generates an **elimination principle**:

$$\texttt{Or.rec} : (A \to C) \to (B \to C) \to A \vee B \to C$$

Disjunction as an Inductive Type

We can *define* $A \vee B$ by declaring its constructors:

```
inductive Or (A B : Prop) : Prop where
  | inl : A → Or A B
  | inr : B → Or A B
```

This is an **inductive definition**: we list all the ways to build an element.

The system automatically generates an **elimination principle**:

$$\texttt{Or.rec} : (A \to C) \to (B \to C) \to A \vee B \to C$$

Key idea: we will use this same mechanism to define natural numbers, equality, and much more.

Natural Numbers as an Inductive Type

Every natural number is either zero, or the successor of another:

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

Natural Numbers as an Inductive Type

Every natural number is either zero, or the successor of another:

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

So $0 = \mathrm{zero}$, $1 = \mathrm{succ}(\mathrm{zero})$, $2 = \mathrm{succ}(\mathrm{succ}(\mathrm{zero}))$, etc.

Natural Numbers as an Inductive Type

Every natural number is either zero, or the successor of another:

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

So $0 = \mathrm{zero}$, $1 = \mathrm{succ}(\mathrm{zero})$, $2 = \mathrm{succ}(\mathrm{succ}(\mathrm{zero}))$, etc.

This corresponds to the idea of a **recursively defined set**:

- $0 \in \mathbb{N}$
- If $n \in \mathbb{N}$, then $\mathrm{succ}(n) \in \mathbb{N}$
- Nothing else is in $\mathbb{N}$

Natural Numbers as an Inductive Type

Every natural number is either zero, or the successor of another:

```
inductive Nat : Type where
  | zero : Nat
  | succ : Nat → Nat
```

So $0 = \mathrm{zero}$, $1 = \mathrm{succ}(\mathrm{zero})$, $2 = \mathrm{succ}(\mathrm{succ}(\mathrm{zero}))$, etc.

This corresponds to the idea of a **recursively defined set**:

- $0 \in \mathbb{N}$
- If $n \in \mathbb{N}$, then $\mathrm{succ}(n) \in \mathbb{N}$
- Nothing else is in $\mathbb{N}$

The "nothing else" is important: it gives us a recursion/induction principle.

The Recursion Principle

Because $\mathbb{N}$ is inductively defined, we get a way to define functions out of $\mathbb{N}$ and to prove properties of all natural numbers.

To define $f : \mathbb{N} \to C$, it suffices to give:

- A value $f(\mathrm{zero}) : C$ (base case)
- A function: given n and $f(n)$, produce $f(\mathrm{succ}(n)) : C$ (step)

The Recursion Principle

Because $\mathbb{N}$ is inductively defined, we get a way to define functions out of $\mathbb{N}$ and to prove properties of all natural numbers.

To define $f : \mathbb{N} \to C$, it suffices to give:

- A value $f(\mathrm{zero}) : C$ (base case)
- A function: given n and $f(n)$, produce $f(\mathrm{succ}(n)) : C$ (step)

To prove $\forall n : \mathbb{N},\ P(n)$, it suffices to show:

- $P(\mathrm{zero})$ (base case)
- $\forall n,\ P(n) \to P(\mathrm{succ}(n))$ (inductive step)

The Recursion Principle

Because $\mathbb{N}$ is inductively defined, we get a way to define functions out of $\mathbb{N}$ and to prove properties of all natural numbers.

To define $f : \mathbb{N} \to C$, it suffices to give:

- A value $f(\mathrm{zero}) : C$ (base case)
- A function: given n and $f(n)$, produce $f(\mathrm{succ}(n)) : C$ (step)

To prove $\forall n : \mathbb{N},\ P(n)$, it suffices to show:

- $P(\mathrm{zero})$ (base case)
- $\forall n,\ P(n) \to P(\mathrm{succ}(n))$ (inductive step)

The type of the recursor (modulo quantification over P):

$$\mathrm{Nat.rec} : P(\mathrm{zero}) \to (\forall n,\ P(n) \to P(\mathrm{succ}(n))) \to \forall n,\ P(n)$$

Pattern Matching

In Lean 4, we can define functions on ℕ by **pattern matching**:

```
def isZero (n : Nat) : Bool := match n with
  | Nat.zero   => true
  | Nat.succ _ => false
```

Pattern Matching

In Lean 4, we can define functions on ℕ by **pattern matching**:

```
def isZero (n : Nat) : Bool := match n with
  | Nat.zero   => true
  | Nat.succ _ => false
```

Under the hood, `match` is **desugared** into applications of the recursor `Nat.rec` (or variations of it).

The recursor is the fundamental primitive; pattern matching is syntactic sugar.

Defining Addition

We define addition by recursion on the **first** argument:

```
def add (n m : Nat) : Nat :=
  match n with
  | Nat.zero   => m                    -- (1)
  | Nat.succ n => Nat.succ (add n m)   -- (2)
```

Defining Addition

We define addition by recursion on the **first** argument:

```
def add (n m : Nat) : Nat :=
  match n with
  | Nat.zero   => m                    -- (1)
  | Nat.succ n => Nat.succ (add n m)   -- (2)
```

This means:

- $\mathrm{add}(\mathrm{zero}, m) = m$ (by equation (1))
- $\mathrm{add}(\mathrm{succ}(n), m) = \mathrm{succ}(\mathrm{add}(n, m))$ (by equation (2))

Defining Addition

We define addition by recursion on the **first** argument:

```
def add (n m : Nat) : Nat :=
  match n with
  | Nat.zero   => m                      -- (1)
  | Nat.succ n => Nat.succ (add n m)     -- (2)
```

This means:

- $\mathrm{add}(\mathrm{zero}, m) = m$ (by equation (1))
- $\mathrm{add}(\mathrm{succ}(n), m) = \mathrm{succ}(\mathrm{add}(n, m))$ (by equation (2))

Let us compute $\mathrm{add}(\mathrm{succ}(\mathrm{succ}(\mathrm{zero})),\ \mathrm{succ}(\mathrm{zero}))$:

$$\begin{aligned}
\mathrm{add}(\mathrm{succ}(\mathrm{succ}(\mathrm{zero})),\ \mathrm{succ}(\mathrm{zero})) &= \mathrm{succ}(\mathrm{add}(\mathrm{succ}(\mathrm{zero}), \mathrm{succ}(\mathrm{zero}))) && \text{(by 2)} \\
&= \mathrm{succ}(\mathrm{succ}(\mathrm{add}(\mathrm{zero}, \mathrm{succ}(\mathrm{zero})))) && \text{(by 2)} \\
&= \mathrm{succ}(\mathrm{succ}(\mathrm{succ}(\mathrm{zero}))) && \text{(by 1)}
\end{aligned}$$

What Can We Prove “For Free”?

Since $\mathrm{add}(\mathrm{zero}, m) = m$ is an equation in the definition, the system knows it **by computation** (definitional equality):

$$\mathrm{add}(\mathrm{zero}, m) =_{\mathrm{def}} m \qquad \text{— ”Definitional equality”}$$

What Can We Prove "For Free"?

Since $\mathrm{add}(\mathrm{zero}, m) = m$ is an equation in the definition, the system knows it **by computation** (definitional equality):

$$\mathrm{add}(\mathrm{zero}, m) =_{\mathrm{def}} m \qquad \text{— "Definitional equality"}$$

But what about $\mathrm{add}(n, \mathrm{zero}) =_{\mathrm{def}} n$?

The definition recurses on the *first* argument. When the second argument is zero, neither equation directly simplifies:

- Equation (1): $\mathrm{add}(\mathrm{zero}, \mathrm{zero}) = \mathrm{zero}$ ✓ (but only for $n = \mathrm{zero}$)
- Equation (2): $\mathrm{add}(\mathrm{succ}(n), \mathrm{zero}) = \mathrm{succ}(\mathrm{add}(n, \mathrm{zero}))$ (still has $\mathrm{add}(n, \mathrm{zero})$!)

What Can We Prove "For Free"?

Since $\mathrm{add}(\mathrm{zero}, m) = m$ is an equation in the definition, the system knows it **by computation** (definitional equality):

$$\mathrm{add}(\mathrm{zero}, m) =_{\mathrm{def}} m \qquad \text{— "Definitional equality"}$$

But what about $\mathrm{add}(n, \mathrm{zero}) =_{\mathrm{def}} n$?

The definition recurses on the *first* argument. When the second argument is zero, neither equation directly simplifies:

- Equation (1): $\mathrm{add}(\mathrm{zero}, \mathrm{zero}) = \mathrm{zero}$ ✓ (but only for $n = \mathrm{zero}$)
- Equation (2): $\mathrm{add}(\mathrm{succ}(n), \mathrm{zero}) = \mathrm{succ}(\mathrm{add}(n, \mathrm{zero}))$ (still has $\mathrm{add}(n, \mathrm{zero})$!)

We need to prove this **by induction** on n. But first, we need to define what **equality** means.

Leibniz Equality: Definition and Reflexivity

Idea (Leibniz): x and y are equal if every property of x is also a property of y.

We write $x =_L y$ for this notion. Formally:

$$x =_L y \;:=\; \forall (P : S \to \mathrm{Prop}),\; P\,x \to P\,y$$

Leibniz Equality: Definition and Reflexivity

Idea (Leibniz): x and y are equal if every property of x is also a property of y.

We write $x =_L y$ for this notion. Formally:

$$x =_L y \;:=\; \forall (P : S \to \mathrm{Prop}),\; P\,x \to P\,y$$

Reflexivity: $x =_L x$ holds.

- We need: for any P, given $P\,x$, produce $P\,x$.
- Proof: $\lambda P.\ \lambda h.\ h$ (the identity!)

Leibniz Equality: Symmetry

Symmetry: $x =_L y \to y =_L x$.

Leibniz Equality: Symmetry

Symmetry: $x =_L y \to y =_L x$.

- Given $e : x =_L y$, i.e. $e : \forall P,\ P\,x \to P\,y$
- Instantiate $P := \lambda z.\ z =_L x$
- Then $e\,P : (x =_L x) \to (y =_L x)$
- We know $x =_L x$ (reflexivity), so we get $y =_L x$ $\square$

Leibniz Equality: Symmetry

Symmetry: $x =_L y \to y =_L x$.

- Given $e : x =_L y$, i.e. $e : \forall P,\ P\,x \to P\,y$
- Instantiate $P := \lambda z.\ z =_L x$
- Then $e\,P : (x =_L x) \to (y =_L x)$
- We know $x =_L x$ (reflexivity), so we get $y =_L x$ $\square$

Congruence: $x =_L y \to f(x) =_L f(y)$ for any function f.

Leibniz Equality: Symmetry

Symmetry: $x =_L y \to y =_L x$.

- Given $e : x =_L y$, i.e. $e : \forall P,\ P\,x \to P\,y$
- Instantiate $P := \lambda z.\ z =_L x$
- Then $e\,P : (x =_L x) \to (y =_L x)$
- We know $x =_L x$ (reflexivity), so we get $y =_L x$ $\square$

Congruence: $x =_L y \to f(x) =_L f(y)$ for any function f.

- Given $e : \forall P,\ P\,x \to P\,y$
- We need: $\forall Q,\ Q\,(f\,x) \to Q\,(f\,y)$
- Instantiate e with $P := \lambda z.\ Q\,(f\,z)$
- Then $e\,P\,h : Q\,(f\,y)$ $\square$

Proof by Induction: $\mathrm{add}(n, 0) =_L n$

Theorem. $\forall n : \mathbb{N},\ \mathrm{add}(n, \mathrm{zero}) =_L n$.

Proof by induction on n:

Proof by Induction: $\mathrm{add}(n, 0) =_L n$

Theorem. $\forall n : \mathbb{N},\ \mathrm{add}(n, \mathrm{zero}) =_L n$.

Proof by induction on n:

Base case ($n = \mathrm{zero}$):

$$\mathrm{add}(\mathrm{zero}, \mathrm{zero}) =_L \mathrm{zero} \qquad \text{by equation (1) — reflexivity.}$$

Proof by Induction: $\mathrm{add}(n, 0) =_L n$

Theorem. $\forall n : \mathbb{N},\ \mathrm{add}(n, \mathrm{zero}) =_L n$.

Proof by induction on n:

Base case ($n = \mathrm{zero}$):

$$\mathrm{add}(\mathrm{zero}, \mathrm{zero}) =_L \mathrm{zero} \qquad \text{by equation (1) — reflexivity.}$$

Inductive step ($n = \mathrm{succ}(k)$, assuming $\mathrm{add}(k, \mathrm{zero}) =_L k$):

$$\begin{aligned} \mathrm{add}(\mathrm{succ}(k), \mathrm{zero}) &=_L \mathrm{succ}(\mathrm{add}(k, \mathrm{zero})) && \text{(def eq (2) + reflexivity)} \\ &=_L \mathrm{succ}(k) && \text{(induction hypothesis and Leibniz \textbf{congruence})} \end{aligned}$$

The Proof in Lean

Using the **recursor** directly (a term-mode proof):

```
Nat.rec (eqL_refl 0) (fun k ih => eqL_congr Nat.succ ih)
```

The Proof in Lean

Using the **recursor** directly (a term-mode proof):

```
Nat.rec (eqL_refl 0) (fun k ih => eqL_congr Nat.succ ih)
```

Reading this term:

- Base case: `eqL_refl 0` proves $\mathrm{add}(\mathrm{zero}, \mathrm{zero}) =_L \mathrm{zero}$ (reflexivity)
- Step: `eqL_congr Nat.succ ih` applies congruence to the induction hypothesis `ih` : $\mathrm{add}(k, \mathrm{zero}) =_L k$, yielding $\mathrm{succ}(\mathrm{add}(k, \mathrm{zero})) =_L \mathrm{succ}(k)$

The Proof in Lean

Using the **recursor** directly (a term-mode proof):

```
Nat.rec (eqL_refl 0) (fun k ih => eqL_congr Nat.succ ih)
```

Reading this term:

- Base case: `eqL_refl 0` proves $\mathrm{add}(\mathrm{zero}, \mathrm{zero}) =_L \mathrm{zero}$ (reflexivity)
- Step: `eqL_congr Nat.succ ih` applies congruence to the induction hypothesis `ih` : $\mathrm{add}(k, \mathrm{zero}) =_L k$, yielding $\mathrm{succ}(\mathrm{add}(k, \mathrm{zero})) =_L \mathrm{succ}(k)$

This is a direct application of `Nat.rec`: induction is just recursion!

Inductive Equality

Lean does not use Leibniz equality. Instead, equality is defined as ...

Inductive Equality

Lean does not use Leibniz equality. Instead, equality is defined as … an inductive type!

```
inductive Eq (a : α) : α → Prop where
  | refl : Eq a a
```

Inductive Equality

Lean does not use Leibniz equality. Instead, equality is defined as … an inductive type!

```
inductive Eq (a : α) : α → Prop where
  | refl : Eq a a
```

This is Lean's built-in equality: when we write $x = y$, this is what it means. Not definitional equality ($=_{\text{def}}$), not Leibniz equality ($=_L$), but this family of inductive types.

Inductive Equality

Lean does not use Leibniz equality. Instead, equality is defined as … an inductive type!

```
inductive Eq (a : α) : α → Prop where
  | refl : Eq a a
```

This is Lean's built-in equality: when we write $x = y$, this is what it means. Not definitional equality ($=_{\text{def}}$), not Leibniz equality ($=_L$), but this family of inductive types.

The elimination principle (`Eq.rec`), modulo quantification over P and a:

$$\texttt{Eq.rec} : P\,a \to \forall\,(b : \alpha),\ a = b \to P\,b$$

This looks a lot like Leibniz equality "in reverse"!

Connecting the Two Equalities

Leibniz: $x =_L y := \forall P,\ P\,x \to P\,y$

Inductive: $x = y$ with constructor $\mathrm{refl} : x = x$, and eliminator $\mathrm{Eq.rec} : P\,x \to x = y \to P\,y$.

Connecting the Two Equalities

Leibniz: $x =_L y := \forall P,\ P\,x \to P\,y$

Inductive: $x = y$ with constructor $\mathrm{refl} : x = x$, and eliminator $\mathrm{Eq.rec} : P\,x \to x = y \to P\,y$.

Exercise: Show that the two notions are logically equivalent:

- Inductive $\Rightarrow$ Leibniz: given $e : x = y$, produce $\forall P,\ P\,x \to P\,y$.
- Leibniz $\Rightarrow$ Inductive: given $e : \forall P,\ P\,x \to P\,y$, produce $x = y$.

Connecting the Two Equalities

Leibniz: $x =_L y := \forall P,\ P\,x \to P\,y$

Inductive: $x = y$ with constructor $\mathrm{refl} : x = x$, and eliminator $\mathrm{Eq.rec} : P\,x \to x = y \to P\,y$.

Exercise: Show that the two notions are logically equivalent:

- Inductive $\Rightarrow$ Leibniz: given $e : x = y$, produce $\forall P,\ P\,x \to P\,y$.
- Leibniz $\Rightarrow$ Inductive: given $e : \forall P,\ P\,x \to P\,y$, produce $x = y$.

(Hint: for one direction, use $\mathrm{Eq.rec}$ directly. For the other, choose a clever instantiation of P.)

Prop vs Type (Preview)

Or and Eq live in Prop, while $\mathbb{N}$ lives in Type. Lean distinguishes the two:

- Prop: the universe of propositions (things to prove)
- Type: the universe of data types (things to compute with)

Prop vs Type (Preview)

Or and Eq live in Prop, while $\mathbb{N}$ lives in Type. Lean distinguishes the two:

- Prop: the universe of propositions (things to prove)
- Type: the universe of data types (things to compute with)

Key differences (maybe more in a later lecture, maybe not):

- **Proof irrelevance:** if $h_1, h_2 : P$ for $P : \mathrm{Prop}$, then $h_1 = h_2$. All proofs of a proposition are considered equal.
- **No large elimination from Prop:** you cannot extract data from a proof. A proof of $A \vee B$ does not let you compute which case holds.

Prop vs Type (Preview)

Or and Eq live in Prop, while $\mathbb{N}$ lives in Type. Lean distinguishes the two:

- Prop: the universe of propositions (things to prove)
- Type: the universe of data types (things to compute with)

Key differences (maybe more in a later lecture, maybe not):

- **Proof irrelevance:** if $h_1, h_2 : P$ for $P : \mathrm{Prop}$, then $h_1 = h_2$. All proofs of a proposition are considered equal.
- **No large elimination from Prop:** you cannot extract data from a proof. A proof of $A \vee B$ does not let you compute which case holds.

For now: when defining logical connectives, use Prop. When defining data, use Type.

Exercises

Exercise 1. Prove transitivity of Leibniz equality:
$x =_L y \to y =_L z \to x =_L z$

Exercise 2. Define equality as an inductive type yourself (with just `refl`). Derive symmetry and transitivity from the recursor.

Exercise 3. Prove $\forall n\ m : \mathbb{N},\ \mathrm{add}(\mathrm{succ}(n), m) = \mathrm{succ}(\mathrm{add}(n, m))$.
(Hint: immediate from the definition.)

Exercise 4. Prove $\forall n\ m : \mathbb{N},\ \mathrm{add}(n, \mathrm{succ}(m)) = \mathrm{succ}(\mathrm{add}(n, m))$.
(Hint: requires induction.)

Exercise 5 (harder). Prove commutativity: $\forall n\ m,\ \mathrm{add}(n, m) = \mathrm{add}(m, n)$.
(Hint: use Exercises 3 and 4.)

Curry–Howard: What We Have So Far

Logic	Type theory	Proof/term
$A \to B$	function type $A \to B$	$\lambda(h : A).\ \ldots$
$A \wedge B$	product $A \times B$	(a, b)
$A \vee B$	sum / inductive with $\mathrm{inl}, \mathrm{inr}$	case analysis
$\forall x : S,\ P(x)$	dependent function $\prod_{x:S} P(x)$	$\lambda(x : S).\ \ldots$
$\neg A$	$A \to \mathrm{False}$	function to empty type
$x = y$	inductive Eq (one ctor: refl)	refl + $\mathrm{Eq.rec}$
$\mathbb{N}$, etc.	inductive type	recursion principle

Curry–Howard: What We Have So Far

Logic	Type theory	Proof/term
$A \to B$	function type $A \to B$	$\lambda(h : A).\ \ldots$
$A \wedge B$	product $A \times B$	(a, b)
$A \vee B$	sum / inductive with $\mathrm{inl}, \mathrm{inr}$	case analysis
$\forall x : S,\ P(x)$	dependent function $\prod_{x:S} P(x)$	$\lambda(x : S).\ \ldots$
$\neg A$	$A \to \mathrm{False}$	function to empty type
$x = y$	inductive Eq (one ctor: refl)	refl + $\mathrm{Eq.rec}$
$\mathbb{N}$, etc.	inductive type	recursion principle
$\exists x : S,\ P(x)$	???	???

Curry–Howard: What We Have So Far

Logic	Type theory	Proof/term
$A \to B$	function type $A \to B$	$\lambda(h : A).\ \ldots$
$A \wedge B$	product $A \times B$	(a, b)
$A \vee B$	sum / inductive with $\mathrm{inl}, \mathrm{inr}$	case analysis
$\forall x : S,\ P(x)$	dependent function $\prod_{x:S} P(x)$	$\lambda(x : S).\ \ldots$
$\neg A$	$A \to \mathrm{False}$	function to empty type
$x = y$	inductive Eq (one ctor: refl)	refl + $\mathrm{Eq.rec}$
$\mathbb{N}$, etc.	inductive type	recursion principle
$\exists x : S,\ P(x)$	???	???

The Existential Quantifier: Deduction Rules

Introduction: to prove $\exists x : S,\ P(x)$, provide a **witness** $a : S$ and a proof of $P(a)$:

$$\frac{\Gamma \vdash a : S \qquad \Gamma \vdash P(a)}{\Gamma \vdash \exists x : S,\ P(x)}$$

The Existential Quantifier: Deduction Rules

Introduction: to prove $\exists x : S,\ P(x)$, provide a **witness** $a : S$ and a proof of $P(a)$:

$$\frac{\Gamma \vdash a : S \qquad \Gamma \vdash P(a)}{\Gamma \vdash \exists x : S,\ P(x)}$$

Elimination: given $\exists x : S,\ P(x)$ and a way to deduce C from any witness:

$$\frac{\Gamma \vdash \exists x : S,\ P(x) \qquad \Gamma \cup \{z : S,\ P(z)\} \vdash C}{\Gamma \vdash C}$$

The Existential Quantifier: Deduction Rules

Introduction: to prove $\exists x : S,\ P(x)$, provide a **witness** $a : S$ and a proof of $P(a)$:

$$\frac{\Gamma \vdash a : S \qquad \Gamma \vdash P(a)}{\Gamma \vdash \exists x : S,\ P(x)}$$

Elimination: given $\exists x : S,\ P(x)$ and a way to deduce C from any witness:

$$\frac{\Gamma \vdash \exists x : S,\ P(x) \qquad \Gamma \cup \{z : S,\ P(z)\} \vdash C}{\Gamma \vdash C}$$

Compare with $\vee$: same shape! $\exists$ is a disjunction over *all* possible witnesses.

Dependent Pairs

Recall: $\forall x : S,\ P(x)$ corresponds to **dependent functions** $\prod_{x:S} P(x)$: a function that takes $x : S$ and returns a proof of $P(x)$.

Dependent Pairs

Recall: $\forall x : S,\ P(x)$ corresponds to **dependent functions** $\prod_{x:S} P(x)$: a function that takes $x : S$ and returns a proof of $P(x)$.

By analogy, $\exists x : S,\ P(x)$ corresponds to **dependent pairs** $\sum_{x:S} P(x)$:

A pair $(a,\ h)$ where $a : S$ and $h : P(a)$.

Dependent Pairs

Recall: $\forall x : S,\ P(x)$ corresponds to **dependent functions** $\prod_{x:S} P(x)$: a function that takes $x : S$ and returns a proof of $P(x)$.

By analogy, $\exists x : S,\ P(x)$ corresponds to **dependent pairs** $\sum_{x:S} P(x)$:

A pair $(a,\ h)$ where $a : S$ and $h : P(a)$.

The type of h *depends* on the value of a. This is what makes it a **dependent** pair (as opposed to an ordinary pair $A \times B$ where both types are fixed).

Dependent Pairs

Recall: $\forall x : S,\ P(x)$ corresponds to **dependent functions** $\prod_{x:S} P(x)$: a function that takes $x : S$ and returns a proof of $P(x)$.

By analogy, $\exists x : S,\ P(x)$ corresponds to **dependent pairs** $\sum_{x:S} P(x)$:

A pair $(a,\ h)$ where $a : S$ and $h : P(a)$.

The type of h *depends* on the value of a. This is what makes it a **dependent** pair (as opposed to an ordinary pair $A \times B$ where both types are fixed).

	Non-dependent	Dependent	
Functions	$A \to B$	$\prod_{x:A} B(x)$	$(\forall)$
Pairs	$A \times B$	$\sum_{x:A} B(x)$	$(\exists)$

Existential as an Inductive Type

In Lean, ∃ is defined as an inductive type:

```
inductive Exists {S : Sort u} (P : S → Prop) : Prop where
  | intro : (w : S) → P w → Exists P
```

Existential as an Inductive Type

In Lean, ∃ is defined as an inductive type:

```
inductive Exists {S : Sort u} (P : S → Prop) : Prop where
  | intro : (w : S) → P w → Exists P
```

There is one constructor: `intro` takes a witness w and a proof of $P(w)$.

Existential as an Inductive Type

In Lean, ∃ is defined as an inductive type:

```
inductive Exists {S : Sort u} (P : S → Prop) : Prop where
  | intro : (w : S) → P w → Exists P
```

There is one constructor: `intro` takes a witness w and a proof of $P(w)$.

Example: "there exists n such that $n + n = 4$":

```
example : ∃ n : Nat, n + n = 4 := Exists.intro 2 rfl
```

Existential as an Inductive Type

In Lean, ∃ is defined as an inductive type:

```
inductive Exists {S : Sort u} (P : S → Prop) : Prop where
  | intro : (w : S) → P w → Exists P
```

There is one constructor: `intro` takes a witness w and a proof of $P(w)$.

Example: "there exists n such that $n + n = 4$":

```
example : ∃ n : Nat, n + n = 4 := Exists.intro 2 rfl
```

The recursion/elimination principle:

Existential as an Inductive Type

In Lean, ∃ is defined as an inductive type:

```
inductive Exists {S : Sort u} (P : S → Prop) : Prop where
  | intro : (w : S) → P w → Exists P
```

There is one constructor: `intro` takes a witness w and a proof of $P(w)$.

Example: "there exists n such that $n + n = 4$":

```
example : ∃ n : Nat, n + n = 4 := Exists.intro 2 rfl
```

The recursion/elimination principle:

$$\mathrm{Exists.rec} : (\forall w : S,\ P(w) \to C) \to (\exists x,\ P(x)) \to C$$

Given a way to handle *any* witness, we can eliminate the existential.

A More General Pattern of Inductive Types

Many inductive types follow the same schema:

```
inductive T : Sort u where
  | c₁ : A₁₁ → A₁₂ → ... → T     -- constructor 1
  | c₂ : A₂₁ → A₂₂ → ... → T     -- constructor 2
  ...
```

A More General Pattern of Inductive Types

Many inductive types follow the same schema:

```
inductive T : Sort u where
  | c₁ : A₁₁ → A₁₂ → ... → T     -- constructor 1
  | c₂ : A₂₁ → A₂₂ → ... → T     -- constructor 2
  ...
```

This declaration gives us:

1. **Constructors**: the only ways to build a term of type *T*
2. **Recursor** (`T.rec`): to define a function out of *T*, handle each constructor
3. Each constructor argument of type *T* yields an **induction hypothesis**

A More General Pattern of Inductive Types

Many inductive types follow the same schema:

```
inductive T : Sort u where
  | c₁ : A₁₁ → A₁₂ → ... → T      -- constructor 1
  | c₂ : A₂₁ → A₂₂ → ... → T      -- constructor 2
  ...
```

This declaration gives us:

1. **Constructors**: the only ways to build a term of type T
2. **Recursor** (`T.rec`): to define a function out of T, handle each constructor
3. Each constructor argument of type T yields an **induction hypothesis**

The recursor has the shape (for a constructor $\mathrm{c_i} : \underset{\text{non-rec}}{B} \to \underset{\text{rec}}{T} \to \cdots \to T$):

$$\mathrm{T.rec} : \cdots \to \underbrace{(\forall\,(b : B)\,(r : T),\ \underset{\text{IH}}{C\ r} \to \cdots \to C\ (\mathrm{c_i}\ b\ r\ \cdots))}_{\text{case for } \mathrm{c_i}} \to \cdots \to \forall\,(t : T),\ C\ t$$

Examples of Inductive Types

Type	Constructors	# recursive args
False	(none)	—
$A \lor B$	$\mathrm{inl},\ \mathrm{inr}$	0 each
$\mathrm{Eq}\ a$	refl	0
$\exists x,\ P(x)$	intro	0
$\mathbb{N}$	$\mathrm{zero},\ \mathrm{succ}$	0 and 1

Examples of Inductive Types

Type	Constructors	# recursive args
False	(none)	—
$A \vee B$	$\mathrm{inl},\ \mathrm{inr}$	0 each
$\mathrm{Eq}\ a$	refl	0
$\exists x,\ P(x)$	intro	0
$\mathbb{N}$	$\mathrm{zero},\ \mathrm{succ}$	0 and 1

All the logical connectives and data types we have seen are instances of the same mechanism...

Examples of Inductive Types

Type	Constructors	# recursive args
False	(none)	—
$A \vee B$	$\mathrm{inl},\ \mathrm{inr}$	0 each
$\mathrm{Eq}\ a$	refl	0
$\exists x,\ P(x)$	intro	0
$\mathbb{N}$	$\mathrm{zero},\ \mathrm{succ}$	0 and 1

All the logical connectives and data types we have seen are instances of the same mechanism...Or not? (little scam)

Soundness: Not Every “Inductive” Is Safe

Could we define a type whose existence leads to a contradiction?

Soundness: Not Every “Inductive” Is Safe

Could we define a type whose existence leads to a contradiction?

```
inductive Bad : Prop where
  | mk : (Bad → False) → Bad
```

Soundness: Not Every “Inductive” Is Safe

Could we define a type whose existence leads to a contradiction?

```
inductive Bad : Prop where
  | mk : (Bad → False) → Bad
```

The recursor would be:

$$\mathrm{Bad.rec} : ((g : \mathrm{Bad} \to \mathrm{False}) \to C\ (\mathrm{mk}\ g)) \to (b : \mathrm{Bad}) \to C\ b$$

Soundness: Not Every “Inductive” Is Safe

Could we define a type whose existence leads to a contradiction?

```
inductive Bad : Prop where
  | mk : (Bad → False) → Bad
```

The recursor would be:

$$\mathrm{Bad.rec} : ((g : \mathrm{Bad} \to \mathrm{False}) \to C\ (\mathrm{mk}\ g)) \to (b : \mathrm{Bad}) \to C\ b$$

If this were allowed, we could prove False:

- Define $f : \mathrm{Bad} \to \mathrm{False}$ by $f\ (\mathrm{mk}\ g) := g\ (\mathrm{mk}\ g)$
- Then $\mathrm{mk}\ f : \mathrm{Bad}$, so $f\ (\mathrm{mk}\ f) : \mathrm{False}$ — contradiction!

The Strict Positivity Condition

One way to be safe is to enforce a **strict positivity** check:

In a constructor of an inductive type T, the type T may only appear in **strictly positive** positions of the types of the arguments: never to the left of an arrow.

The Strict Positivity Condition

One way to be safe is to enforce a **strict positivity** check:

In a constructor of an inductive type T, the type T may only appear in **strictly positive** positions of the types of the arguments: never to the left of an arrow.

Allowed (positive)	Rejected (negative)
`| succ : Nat → Nat`	`| mk : (Bad → False) → Bad`
`| node : T → T → T`	`| mk : (T → T) → T`
`| inl : A → Or A B`	`| mk : (T → A) → T`

The Strict Positivity Condition

One way to be safe is to enforce a **strict positivity** check:

In a constructor of an inductive type T, the type T may only appear in **strictly positive** positions of the types of the arguments: never to the left of an arrow.

Allowed (positive)	Rejected (negative)
`| succ : Nat → Nat`	`| mk : (Bad → False) → Bad`
`| node : T → T → T`	`| mk : (T → T) → T`
`| inl : A → Or A B`	`| mk : (T → A) → T`

This ensures that every inductive type is well-founded.

Without this check, the type theory would be inconsistent.